

```
// stampa cifre ricorsivamente
function stampaCifreRec(n){
  // caso base
  if (n < 10) {
    console.log(n)
    return
  }
  // caso ricorsivo
  let resto = n % 10
  let quoziente = (n - resto) / 10
  stampaCifreRec(quoziente)
  console.log(resto)
}
```

```
// stampaCifreRec(164392)
```

```
// variante che restituisce un array con le cifre
function arrayCifreRec(n){
  // caso base
  if (n < 10) return [n]
  // caso ricorsivo
  let resto = n % 10
  let quoziente = (n - resto) / 10
  let cifre = arrayCifreRec(quoziente)
  cifre.push(resto)
  return cifre
}
```

```
// console.log(arrayCifreRec(164392))
```

```
// versione con ricorsione in coda
function arrayCifreRecCoda(n, r){ // r rappresenta l'array parziale
  che stiamo riempiendo
  // caso base
  if (n < 10) {
    r.unshift(n)
    return r
  }
  // caso ricorsivo
  let resto = n % 10
  let quoziente = (n - resto) / 10
  r.unshift(resto)
  return arrayCifreRecCoda(quoziente, r)
}
```

```
// console.log(arrayCifreRecCoda(164392, []))
```

```
// Fibonacci: calcolare l'n-simo numero
function fibonacciNesimo(n){
  // caso base
  if (n == 0 || n == 1) return 1
```

```

    // caso ricorsivo
    return fibonacciNesimo(n-1) + fibonacciNesimo(n-2)
}

// console.log(fibonacciNesimo(5))

// Fibonacci: array che contiene i primi n numeri della successione
function fibonacciPrimiN(n){
    // casi base
    if (n == 0)
        return [ 1 ]
    if (n == 1)
        return [ 1, 1 ]
    // caso ricorsivo
    let seqFib = fibonacciPrimiN(n-1)
    let nesimo = seqFib[seqFib.length-1] + seqFib[seqFib.length-2]
    seqFib.push(nesimo)
    return seqFib
}

// console.log(fibonacciPrimiN(5))

// per casa: farlo con ricorsione in coda

// ricerca lineare iterativa -- output booleano
function ricercaLinBool(x,a){
    for (let val of a)
        if (val==x)
            return true
    return false
}

// console.log(ricercaLinBool(5,[1,2,5,3,1,2,6,0]))

// ricerca lineare ricorsiva -- output booleano
// idea: array = [primo, resto]; se il primo elemento non è uguale
// ad el, ricorriamo nel resto dell'array
// tip: slice() ritorna una copia del pezzo indicato dell'array:
// https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/slice; in particolare, slice(1) è l'array meno
// il primo elemento
function ricercaLinRecBool(el,arr){
    // caso base
    if (arr.length == 0) return false
    // caso ricorsivo
    let primo = arr[0]
    if (primo == el) return true
    let resto = arr.slice(1)
    return ricercaLinRecBool(el,resto)
}

```

```

// console.log(ricercaLinRecBool(5,[1,2,5,3,1,2,6,0]))
// console.log(ricercaLinRecBool(45,[1,2,5,3,1,2,6,0]))
// console.log(ricercaLinRecBool(45,[]))

// proviamo adesso a ritornare l'indice di occorrenza dell'elemento

// ricerca lineare iterativa -- indice
function ricercaLin(el,arr){
    for (let i=0; i<arr.length; i++){
        if (arr[i]==el)
            return i
    }
    return -1
}

// console.log(ricercaLin(5,[1,2,5,3,1,2,6,0]))

// ricerca lineare ricorsiva -- indice
function ricercaLinRec(el,arr,currInd=0){ // dobbiamo mantenerci
    quanti elementi abbiamo già scartato per avere l'indice corrente!
    // caso base
    if (arr.length == 0 || currInd == arr.length) return -1
    // caso ricorsivo
    if (arr[currInd] == el) return currInd
    return ricercaLinRec(el,arr,currInd+1)
}

// console.log(ricercaLinRec(5,[1,2,5,3,1,2,6,0]))

// passiamo adesso alla ricerca binaria; prima con output booleano
function ricercaBinBool(el,arr){
    // caso base
    if (arr.length == 0) return false
    // caso ricorsivo
    let centro = Math.floor(arr.length/2)
    if (arr[centro] == el)
        return true
    else if (arr[centro] > el)
        return ricercaBinBool(el,arr.slice(0,centro))
    else
        return ricercaBinBool(el,arr.slice(centro+1))
}

/*
console.log([1,2,5,7,9,12,18,32])
console.log("-5",ricercaBinBool(-5,[1,2,5,7,9,12,18,32]))
console.log("33",ricercaBinBool(33,[1,2,5,7,9,12,18,32]))
console.log("10",ricercaBinBool(10,[1,2,5,7,9,12,18,32]))
console.log("1",ricercaBinBool(1,[1,2,5,7,9,12,18,32]))
console.log("32",ricercaBinBool(32,[1,2,5,7,9,12,18,32]))
console.log("5",ricercaBinBool(5,[1,2,5,7,9,12,18,32]))
*/

```

```

// ricerca binaria -- output indice
function ricercaBin(el,arr,sxInd=0,dxInd=arr.length){ // qui
dobbiamo tenerci gli indici di inizio e fine della partizione
corrente dell'array
    // caso base
    if (sxInd >= dxInd) return -1
    // caso ricorsivo
    let centro = Math.floor((sxInd + dxInd)/2)
    if (arr[centro] == el)
        return centro
    else if (arr[centro] > el)
        return ricercaBin(el,arr,sxInd,centro)
    else
        return ricercaBin(el,arr,centro+1,dxInd)
}

/*
let arr = [1,2,5,7,9,12,18,32]; // arr.length = 8
console.log(ricercaBin(-5,[1,2,5,7,9,12,18,32]))
console.log(ricercaBin(33,[1,2,5,7,9,12,18,32]))
console.log(ricercaBin(10,[1,2,5,7,9,12,18,32]))
console.log(ricercaBin(1,[1,2,5,7,9,12,18,32]))
console.log(ricercaBin(32,[1,2,5,7,9,12,18,32]))
console.log(ricercaBin(5,[1,2,5,7,9,12,18,32]))
*/

// console.log("-----")

```